

 Universidade do Porto Faculdade de Engenharia FEUP	Faculdade de Engenharia da Universidade do Porto Mestrado Integrado em Engenharia Informática e Computação Programação em Lógica <i>Época de Recurso – Com Consulta / Duração: 2h30m (+15 min)</i>	2006/2007 3º Ano MIEIC
	Nome:	Data:

Nota: Utilize exclusivamente Prolog Standard e não utilize nenhuma biblioteca específica a não ser a clpfd no grupo 6.

GRUPO I – Programação em Prolog

1) Ligas de Futebol

Suponha a seguinte base de conhecimento em Prolog sobre as ligas, equipas e jogadores do campeonato português de futebol e respectivos golos marcados.

Exemplo da Base de Factos:

```

lig(super, porto).           jog(porto, quaresma, av, 6).
lig(super, benfica).        jog(porto, helton, gr, 0).
lig(super, sporting).       jog(benfica, simao, av, 7).
lig(honra, feirense).       jog(sporting, moutinho, me, 3).
lig(honra, guimaraes).      jog(sporting, ricardo, gr, 0).
                             jog(guimaraes, cleber, de, 1).
                             jog(feirense, edgar, me, 8).

```

Escreva um predicado em Prolog *jogadores(+Posicao, +Liga, +NGol, -Lista)*, que retorne a Lista de todos os jogadores e respectivos clubes, que jogam numa dada posição, numa dada liga e marcaram NGol golos ou mais. Suponha que quer a posição, quer a liga, quer o NGol pode ser substituído por *, indicando qualquer valor **(2.0 Val.)**.

Exemplo:

```

?- jogadores(av, super, 7, Lista1).
Lista1 = [simao-benfica-7].
?- jogadores(*, honra, *, Lista2).
Lista2 = [cleber-guimaraes-1, edgar-feirense-8].
?- jogadores(*, *, 3, Lista3).
Lista3 = [quaresma-porto-6, simao-benfica-7, moutinho-sporting-3,
edgar-feirense-8].

```

2) Problema da Grelha

Suponha que uma grelha de NxN quadrículas numeradas (como mostra a figura para uma grelha 8x8) é representada como uma lista de listas.

2.1) Pretende-se determinar qual o valor que está numa quadrícula que se encontra numa dada direcção (ci, ba, es, di) relativamente a uma outra quadrícula. Construa um predicado *quadrícula_direcção(+Grelha, +X/Y, +Dir, -Xf/Yf, -Val)*, que determine a posição Xf/Yf e o valor da quadrícula que se encontra numa dada direcção relativamente à quadrícula X/Y. **(1.0 Val.)**

Exemplo:

```

quadrícula_direcção([[1,2,2,2,1,1,5,5],[3,2,
... 4,2,2,1,1]], 5/3, ci, Xf/Yf, Val).
?- Xf = 5, Yf = 2, Val1 = 4
quadrícula_direcção([[1,2,2,2,1,1,5,5],[3,2,
... 4,2,2,1,1]], 5/3, es, Xf/Yf, Val).
?- Xf = 4, Yf = 3, Val1 = 1

```

Y/X	1	2	3	4	5	6	7	8
1	1	2	2	2	1	1	5	5
2	3	2	3	2	4	1	5	5
3	3	3	1	1	2	1	5	5
4	3	3	1	1	2	5	3	5
5	4	4	1	1	2	2	3	5
6	5	5	5	2	2	2	5	5
7	4	4	3	4	2	2	1	2
8	4	4	3	4	2	2	1	1

2.2) Pretende agora descobrir células isoladas (i.e.) que não tenham nenhuma célula idêntica vizinha, na horizontal ou vertical. Construa o predicado *celula_isolada(+Grelha, ?X/Y)*, que lhe permite determinar se uma dada célula está isolada ou não. **(1.0 Val.)**

Exemplo:

```
celula_isolada([ [1,2,2,2,1,1,5,5], [3,2,... 4,2,2,1,1] ], 5/2) .
?- yes.
celula_isolada([ [1,2,2,2,1,1,5,5], [3,2,... 4,2,2,1,1] ], 7/2) .
?- no.
celula_isolada([ [1,2,2,2,1,1,5,5], [3,2,... 4,2,2,1,1] ], X/Y) .
?- X = 1, Y = 1 ;
?- X = 3, Y = 2 ;
...
```

2.3) Suponha agora que pretende descobrir todas as células que estão ligadas a uma dada célula. Construa um predicado eficiente designado *conjunto_celulas(+Grelha, +X/Y, -ListaC)* que calcule todas as células ligadas a uma dada célula. **(2.0 Val.)**

Exemplo:

```
conjunto_celulas([ [1,2,2,2,1,1,5,5], [3,... 2,1,1] ], 1/1, ListaC) .
?- ListaC = [].
conjunto_celulas([ [1,2,2,2,1,1,5,5], [3,... 2,1,1] ], 5/5, ListaC) .
?- ListaC = [5/4, 5/3, 6/5, 5/6, 6/6, 4/6, 5/7, 6/7, 5/8, 6/8] .
```

3) Intersecção de Conjuntos

Implemente em Prolog um procedimento para calcular a intersecção de dois conjuntos de intervalos de números inteiros. Cada conjunto de intervalos é codificado usando uma lista de números inteiros em que cada par de números consecutivos representa o limite do intervalo (inclusive). Considere que os números nas listas estão ordenados por ordem crescente. O predicado a implementar aceita os dois conjuntos de intervalos nos 2 primeiros argumentos e devolve a sua intersecção no terceiro, no mesmo formato. **(2.5 Val.)**

Exemplo:

```
?- interseccao([2, 4, 6, 10], [1, 3, 10, 10], Inters) .
Inters = [2, 3, 10, 10]
?- interseccao([2, 4, 5, 8, 9, 12], [1, 11], Inters) .
Inters = [2, 4, 5, 8, 9, 11]
```

4) Predicado Qualquer

Considere que tem disponível no Prolog o predicado *qualquer(+X, +Y, -Z)*. Assuma que quando chama este predicado os dois primeiros argumentos devem estar instanciados e o terceiro argumento é uma variável. Implemente em Prolog um predicado *"coleccionaRespostas"* que recebe como primeiro e segundo argumentos duas listas com valores possíveis para os dois primeiros argumentos do predicado *"qualquer/3"* e devolve como terceiro argumento uma lista de triplos representando o resultado de todas as chamadas possíveis a *"qualquer/3"*, incluindo os valores fornecidos para os seus argumentos e o resultado. **(2.0 Val.)**

Exemplo, supondo: *qualquer(A,B,C) :- C is A*B.*

```
?- coleccionaRespostas([1,5], [3,4], Resultado) .
Resultado = [[1,3,3], [1,4,4], [5,3,15], [5,4,20]] .
```

5) Perguntas Teórico-Práticas

Verifique se os seguintes termos são unificáveis em Prolog. Justifique cada uma das respostas, indicando os resultados das unificações válidas e as razões pelo que as restantes não são válidas **(2.0 Val.)**.

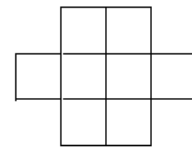
- | | |
|---|----------------------------------|
| a) <i>x(a, 2)</i> | e <i>x(B, C)</i> |
| b) <i>y(2, A1)</i> | e <i>y(A2, 3)</i> |
| c) <i>z(2, A)</i> | e <i>z(A, 3)</i> |
| d) <i>xpto(f(g(R)), R)</i> | e <i>xpto(f(U), g(V))</i> |
| e) <i>term(Y, g(f(X)), f(Z))</i> | e <i>term(f(X), g(r(Z)), Y)</i> |
| f) <i>eu(a, h(X, Y), f(X, a, Y), Z)</i> | e <i>eu(W, V, f(V, V, U), U)</i> |
| g) <i>porto(t(X, t(X, b)))</i> | e <i>porto(t(a, t(Z, Z)))</i> |
| h) <i>w(Z, f(X, a, Y), h(X, Y), a)</i> | e <i>w(W, f(V, V, U), W, U)</i> |

GRUPO II – Programação em Lógica com Restrições

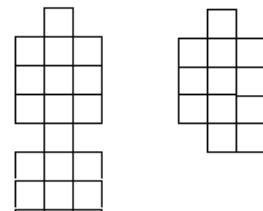
6.1) Considere o seguinte puzzle em que o objectivo consiste em colocar as 8 primeiras letras do alfabeto sem que duas letras seguidas no alfabeto estejam em quadrículas adjacentes (na horizontal, vertical ou diagonal). Resolva este problema utilizando Programação em Lógica com Restrições. **(1.5 val)**.

```
?- resolve(Sol).  
Sol = [c, e, g, a, h, b, d, f] ;  
Sol = [e, c, b, h, a, g, f, d] ;  
...
```

A B C D E F G H



6.2) Suponha que desejava construir um programa para resolver puzzles, semelhantes aos da alínea anterior mas com formas arbitrárias (por exemplo as da figura ao lado) e em que possa especificar, para cada letra, uma lista de letras em que não deve tocar e (em vez de não dever tocar só nas letras anterior e posterior do alfabeto). Suponha que deseja obter todas as soluções possíveis para um dado puzzle e não somente a primeira. Como estruturaria os dados? Altere o programa anterior para permitir resolver este novo problema. **(1.5 val)**.



6.3) Considere que dispõe de 3 processadores para executar um conjunto de tarefas. Elabore um predicado que recebe a lista de tarefas a executar como primeiro argumento e devolve uma lista com três elementos em que o primeiro elemento é uma lista com as tarefas a executar no primeiro processador, o segundo elemento é a lista com as tarefas para o segundo processador e o terceiro elemento é a lista com as tarefas para o terceiro processador. Cada elemento da lista de tarefas de entrada é um par *NumeroTarefa/TempoExecucao*. O objectivo é minimizar o tempo total de execução do conjunto das tarefas. **(2.0 val)**.

Exemplo:

```
?- distribui([1/2, 2/3, 3/5, 4/4, 5/1], Distrib).  
Distrib = [[1, 2], [3], [4, 5]].
```

6.4) Suponha que o número de máquinas não é fixo e que são adicionadas restrições de capacidades específicas ou máxima de tarefas executáveis nas máquinas e restrições de impossibilidade de certas máquinas executarem certas tarefas. Encontram-se disponíveis também os dados relativos ao custo de executar cada tarefa em cada máquina e alguns dados relativos a custos conjuntos de execução de um par de tarefas na mesma máquina (que são adições ao custo individual). Suponha que os dados são apresentados no formato seguinte:

```
maquinas(10). // dispomos de 10 máquinas  
maximo(1,2). // maquina 1 só pode executar, no máximo, 2 tarefas  
maximo(4,3).  
especifica(5,2). // máquina 5 executa exactamente duas tarefas  
especifica(7,3).  
impossivel(1, [4,8]). //máquina 1 não pode fazer as tarefas 4 nem 8  
impossivel(3, [2,6]).  
custo(1, 1, 3).  
custo(1, 2, 3). //executar a tarefa 2 na máquina 1 custa 3  
custo(1, 3, 1).  
custo(1, 4, 5).  
...  
custo(10, 25, 2). // executar a tarefa 25 na máquina 10 custa 2  
custo_conj(3, [1,6], 7). // executar as tarefas 1 e 6 ambas na //  
// máquina 3 tem um acrescimo de custo de 7.  
custo_conj(6, [1,2,3,6], 3).  
...
```

Pretende-se determinar e imprimir, devidamente formatada, no ecrã, duas soluções alternativas: mínimo custo e mínima duração. Construa um programa semelhante ao da alínea anterior capaz de resolver esta nova versão mais elaborada do problema disponibilizando as duas soluções desejadas. **(2.5 val)**.